1 General Information

1.1 Document Release

Author:

Role	Name	Date	Signature
Senior QA Engineer	Ammar Khallouf		

Reviewer:

Role	Name	Date	Signature
Senior QA Engineer / Team Lead	TBD		

Approver:

Role	Name	Date	Signature
Head of Department of Author	TBD		

1.2 Table of Contents

1.3 Purpose

The purpose of this document is to describe and outline the quality and validation strategy for the thermal control subsystem of a wearable/IoT device. The focus is on firmware quality, hardware-software integration and edge-case handling. The strategy targets a late-stage product team where focus will be on establishing confidence in subsystem stability, prioritizing high risk testing, scaling test automation with enforced quality gates using CI/CD.

1.4 Change History

Change Request #	Version	Change Description
N.A	1	Initial version

1.5 Reference Documents

[1] DXXXXXX – Thermal Subsystem Architecture Report

[2] DXXXXXX – Thermal Subsystem FW Requirements Specification

[3] DXXXXXX – Thermal Subsystem FW Design Specification

[4] DXXXXXX – QMS Defect Management Plan / Process

1.6 Abbreviations

CCB	Change Control Board
CI / CD	Continuous Integration / Continuous Delivery
CR	Change Request
CRC	Cyclic Redundancy Check
DR	Design Review
HW	Hardware
SW	Software
FW	Firmware
PWM	Pulse Width Modulation
RPM	Revolutions Per Minute
RCA	Root Cause Analysis
I2C	Inter-Integrated Circuit
QA	Quality Assurance
QMS	Quality Management System
MCU	Micro Controller Unit
UART	Universal Asynchronous Receiver/Transmitter

2 Firmware Control Loop and Expected Behaviors

2.1 Overview

The subsystem consists of a PWM fan with a tachometer feedback pin, I2C based temperature sensor and MCU.

The MCU uses PWM signal to control the speed of the fan. By varying the duty cycle of the PWM signal, the fan can operate at different speeds to achieve the desired level of cooling and hold the target temperature. The tachometer pin signal reading allows the MCU to measure the actual RPM speed of the fan to make sure it is closely tracking the set point RPM speed for the target temperature. Hence, the tachometer pin readings allow the MCU to verify the fan operational speed and detect faults (e.g. the fan stops spinning or measured RPM speed deviates from the set point RPM speed).

2.2 Firmware Control Loop and Expected Behaviors

The MCU FW control loop constitutes a closed-loop control system with temperature being the feedback mechanism (i.e. Input to the system) dynamically adjusting the output PWM signal (i.e. control signal setting the speed of the fan). The control loop logic flow is depicted in figure (1) below and could be summarized as per the following steps:

- The FW periodically reads the temperature sensor data over I2C
- The FW validates temperature reading against a valid temperature range
- The FW control algorithm compares the read temperature with the target temperature (setpoint)
- Based on the control algorithm implemented, the required RPM speed of the fan is calculated (represented in the form of PWM duty cycle)
- The FW commands the new PWM duty cycle to the fan
- The FW monitors the tachometer pin readings to calculate actual RPM speed of the fan and compares it to the requested RPM speed
- The FW reports an error state in case the difference between the requested and calculated RPM speed is outside the acceptable tolerance limits
- The FW detects and reports critical issues related to the PWM fan (e.g. no rotation based on zero PWM signal) and temperature sensor (e.g. invalid readings, sensor sent NAK due to I2C communication error)
- A fallback fail-safe mechanism is executed by the FW in case the temperature readings are invalid, tachometer feedback pin reports no rotation, or invalid PWM duty cycles are commanded due to an issue in the control algorithm.

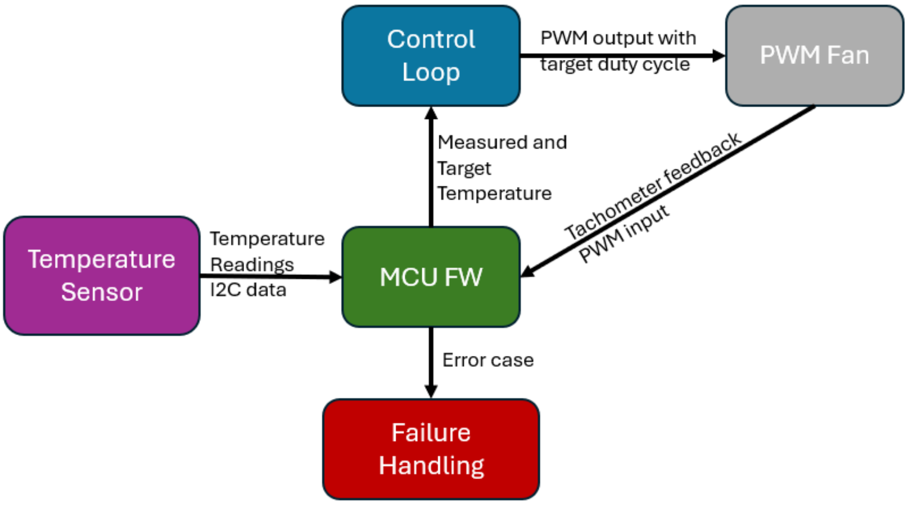

Figure 1: Simplified control logic flow of the thermal control subsystem

The QA engineer needs to go through the firmware control loop sequence described above together with an architect / developer from the product team, such that the understanding of the firmware behavior is common. In this regard, the thermal control subsystem architecture report [2] needs to be reviewed in conjunction with the FW requirements specifications [3] to make sure that acceptance criteria for the developed test cases are well defined (e.g. what is acceptable tolerance for the drift of the tachometer signal?).

With respect to error cases, the thermal control subsystem architecture report [2] should have an error handling section describing the expected error states and corresponding handling mechanisms of the subsystem. (e.g. error codes mapped to specific failures and failsafe handling functions).

Based on the FW logic overview, the following expected behaviors are derived and could be better confirmed by asking the product team to review the below points:

- At system startup or restart and before first valid temperature reading, the FW PWM control signal should have a defined state (e.g. the PWM fan runs at full speed until temperature reading is processed and target PWM duty cycle is calculated).
- The collection of temperature readings from the sensor occurs at the configured time interval. In case the temperature sensor supports CRC checks, failed CRC I2C transactions should be flagged (e.g. CRC error counter is incremented, and FW goes to fail-safe state if counter exceeds given value).
- PWM output control signal monotonically follows temperature (i.e. PWM duty cycle increases with rising temperature and as the system cools temperature drops and the PWM duty cycle decreases).
- The measured RPM speed using the tachometer feedback signal is expected to be within an acceptable tolerance of the expected RPM for the commanded PWM duty cycle.
- Oscillations in the tachometer input PWM signal does not cause false RPM readings (i.e. switching noise causing firmware to interpret it as rotation of the fan).
- The FW control loop should converge (i.e. target temperature reached) without excessive oscillations of the fan speed. For testing it is important to get expected convergence time of the control loop and accepted tolerance limits between target and actual temperature.
- In case of failure scenarios such as broken tachometer and faulty temperature sensor, the FW should detect them and execute a fail-safe strategy.
- For easy testing and diagnosis in the field, the FW is expected to log important events with timestamps (e.g. PWM duty cycle change, temperature, tachometer RPM). Error logs are expected to be generated and saved in case the FW encounters failure scenarios.

3 Test Strategy

The test strategy's input will be the FW requirements specifications [1]. Other valuable inputs could be a risk analysis or safety failure analysis document and the FW design specifications [2]

The testing strategy would prioritize establishing confidence in the FW implementation (functional / logical entity of the subsystem) and the HW-SW integration. Therefore, automated hardware in the loop test cases will be developed to:

- Exercise the FW control loop logic and verify FW response
- Evaluate subsystem response to faults and handling of edge cases
- Evaluate the subsystem behavior over long running duration with a prescribed heat usage profile

The central behavior under test will be the dynamic tracking of the commanded PWM setting by the tachometer signal within an acceptable tolerance (i.e. tachometer signal vs. PWM setting relationship).

To cover edge cases, the response of the subsystem (mainly FW control loop) to disturbances, faulty tachometer and temperature sensor readings will be tested to verify the subsystem's actual behavior against expected behavior outlined in requirements and design specifications [2],[3].

A hardware-in-the-loop, low-cost test station based on Raspberry Pi (Test agent) and Analog Discovery (Logic / Analog Analyzer) will be utilized to support the automated testing of the subsystem. It will be accompanied by a Python test framework with built-in fixtures and drivers to interact with the subsystem under test via PWM, UART, and I2C interfaces.

The test station will be registered as a self-hosted agent with "AzureDevOps" or "GitHub Actions" to run CI/CD workflows with hardware in the loop (e.g. testing tolerance of measured RPM tachometer).

To achieve high scalability and test coverage, multiple low-cost test stations can run different test suites in parallel (e.g. RPi-Station-1: functional/regression tests, RPi-Station-2: stress and performance tests, RPi-Station-3: long runtime tests).

For post-processing of the test results, the last step in CI/CD workflow will perform the following tasks:

- Upload the generated pytest html report to the server (e.g. AzureDevOps, GitHub)
- Publish the test results to a linked test plan
- Analyze the FW error logs and push the data to statistical analysis tool (e.g. Power BI, Grafana)
- Run trace analysis on the latest test and requirements specifications

The QA team would then review the consolidated test results, analyze failure trends, test coverage and present the product team with an overview of critical bugs and quality metrics to evaluate product release readiness.

Failed test cases will be reported and handled according to QMS defect management process in [4]. See Defect Management section for details.

3.1 Testing Levels and Prioritization

In a broad quality strategy multiple layers of testing are executed to catch defects as early and cheaply as possible. For a late-stage of product development lifecycle, execution priority is given to hardware-in-the loop integration and end-to-end system tests with focus on high release-risk use cases that have not been explicitly covered yet. In parallel, stress and performance / long term tests are executed to validate robustness over time and stress. Following bug fixes and new FW builds, regression tests are executed as part of CI workflow.

The suggested testing structure is summarized in the table below:

Risk Priority	Test Level	Test Environment / Automation	Example Test Purpose
1	End-to-End System Tests (High risk use cases)	Automated hardware in the loop test station	Verify FW can detect fan and temperature sensor failures leading to over-temperature condition / undefined control state
2	Parametrized Integration tests	Similar to above	Verify tracking of PWM setting by tachometer signal works on real hardware across the duty range
2	Stress and Performance / Long Term Tests	Similar to above	Verify FW control loop stability under stress, long term behavior for a given set of heating / cooling profiles
3	Regression Tests	Similar to above and can include simulation-based unit/integration tests running only the host	Verify fixed bugs are not reintroduced in future FW builds

3.2 Traceability Matrix

Requirements are mapped to test cases to form a traceability matrix. Each requirement needs to have at least one test case verifying it. The traceability matrix can be automatically generated by running trace analysis against given requirements and test specifications.

Since the focus is on late-stage validation, not all possible traces across all functional requirement levels are included. Instead, a narrowed down matrix targeting important behaviors and high-risk use cases is presented (see section 2.2 for details)

Req ID	Requirement	Test ID(s) - Verifying Test(s)
REQ-1	The FW shall read temperature from I2C sensor periodically according to configured time interval	TEST-1 Test temperature reading interval from sensor
REQ-2	At FW startup, The FW should command the PWM fan to full speed (100% duty cycle) before commanding the target duty cycle that is calculated based on the first valid temperature reading. Similar conditions should apply when the PWM fan transitions from zero commanded speed (0% duty cycle) to above minimum duty cycle of 10%.	TEST-2 Test duty cycle of PWM output signal at FW startup TEST-3 Test duty cycle of PWM output signal when fan resumes spinning from commanded stop state.
REQ-3	The FW control loop shall increase the duty cycle of the output PWM signal as the measured temperature increases above target temperature and it should decrease the PWM duty cycle as the measured temperature decreases toward target temperature.	TEST-4 Test duty cycle of PWM output signal tracks temperature
REQ-4	The measured RPM speed of the fan from tachometer pin matches commanded fan RPM within a ±10% tolerance.	TEST-5 Test Tachometer RPM tracking and accuracy
REQ-5	The FW shall trigger a fault fan feedback error if the measured RPM from the tachometer pin drops below minimum spin threshold for 10 seconds while the commanded PWM output signal is greater than minimum spin threshold (10% duty cycle).	TEST-6 Test faulty non-rotating fan triggers a fault fan feedback error TEST-7 Test zero RPM fault injection handling
REQ-6	The FW shall command a 0% duty cycle for the PWM output signal if the requested duty cycle falls below the specified minimum spin threshold of the fan (10% duty cycle).	TEST-8 Test zero PWM duty cycle commanded below minimum spin threshold
REQ-7	The FW control loop shall dynamically adjust the duty cycle of the PWM output signal such that the measured temperature converges to the target temperature within ±10% tolerance and without causing the PWM output signal to oscillate by more than ±10%	TEST-9 Test FW control loop convergence and stability
REQ-8	The FW shall trigger a thermal feedback error if any of the conditions below occur: • Temperature sensor cannot be detected on the I2C bus within 10 seconds • CRC errors counter exceeds 10 • Temperature sensor readings are outside specified range (including over temperature) [0°C to 55°C]	TEST-10 Test FW error handling of temperature sensor faults
REQ-9	The Firmware shall log diagnostic data (including measured temperature, commanded PWM duty cycle, measured RPM speed from tachometer, detected error logs) and send it over UART when debug mode is enabled in testing.	TEST-11 Test FW diagnostic data read over UART

3.3 Schedule and Timelines

The System test schedule will be part of the product overall plan and follow high-level planning depicted in figure (2) below.

Testing activites ideally start early in parallel to FW implementation and other development activities. The early integration tests are developed based on available drafted/released versions of the requirements specifications. Unit and simulation tests are also run to verify the FW design logic in isolation of the hardware.

Before DR2 phase, HW-SW bringup need to be completed as a a pre-requisite to start end-to-end integration testing with hardware in the loop. With regards to non-functional requirements, a set of measurements are performed to characterize the system behavior and detect any performance bottlnecks (e.g. power consumption and noise measurements, sensor response time, fan speed - PWM duty cycle characterstics).

After DR2, hardware-in-the-loop testing runs to formally verify all the system requirements and introduced bug fixes (whitebox automated testing).

Stress and performance tests run also continously until test activites are completed before DR3. Similarly. Failure mode analysis initiatives are executed to identify testing gaps, possible failure modes and available / required mitigations to address them. Following bug fixing FW releases, regression testing is performed to ensure introduced changes do not break existing functionality and all test cases pass up to and including DR3.

Notes:

- *CW columns in the figure below are rough estimation of planned FW package releases.*
- *DR2 Milestone: FW Feature Complete State*
- *DR3 Milestone: Design Verification*

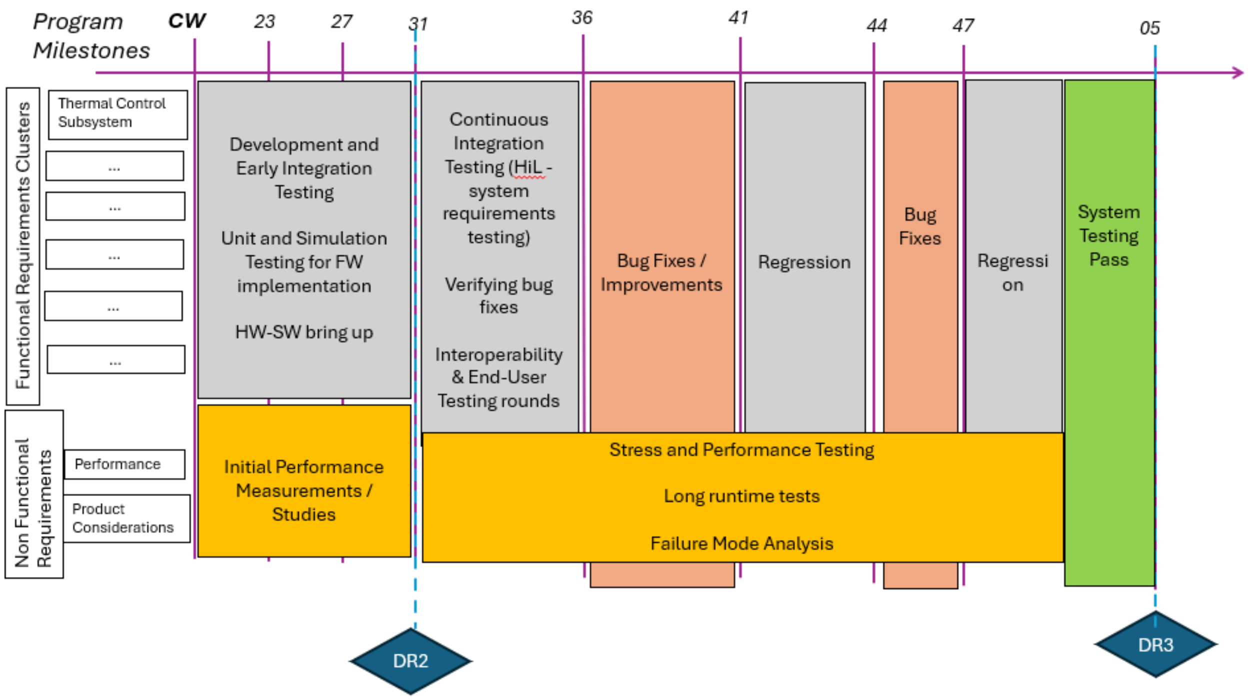

Figure 2: Example product program plan with marked milestones, FW releases and planned test activities

3.4 Defect Management

A defect management process will be in place to handle the lifecycle of defects including identification, investigation, implementation and verification. A severity classification table is also supplemented. For critical defects that escaped lower testing layers, RCA would be conducted to improve FW quality and identify possible gaps in the product development and testing process.

Defects are reported in the form of a CR ticket (Change Request) created and tracked inside a project management tool (e.g. AzureDevOps, JIRA)

A CR (Change Request) has the following state diagram:

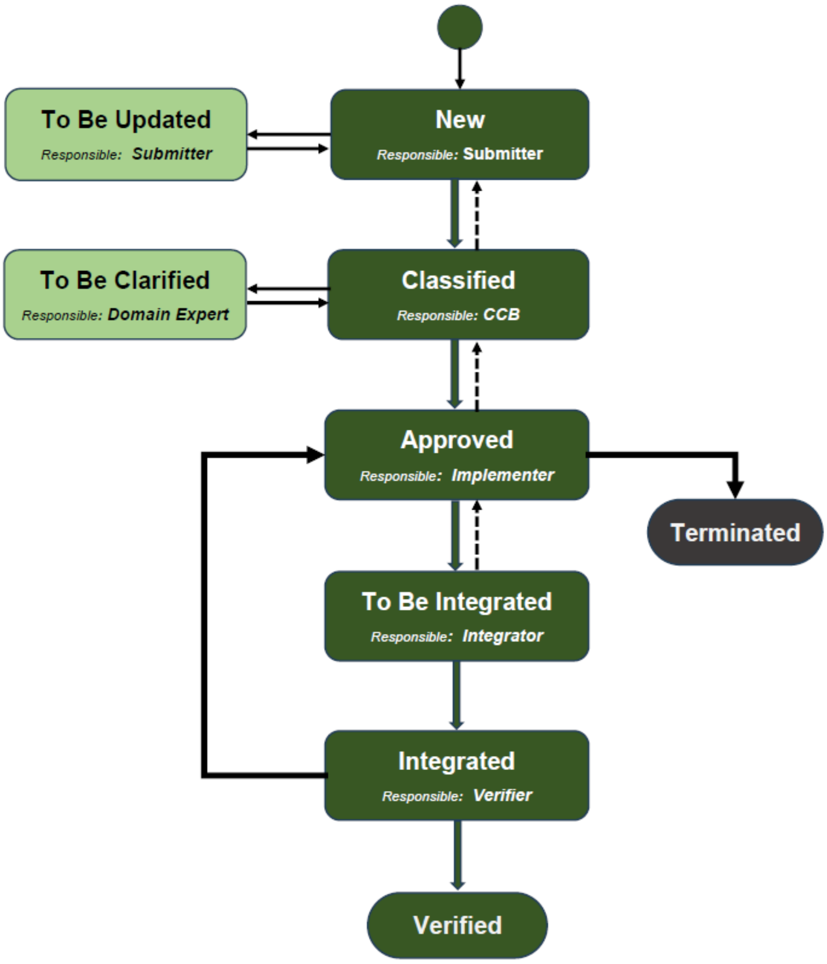

Figure 3: State diagram of Change Request

The CR has initially the state "New" when submitted by the author. Depending on the scope of the CR and project phase, the CR is redirected by the project management tool to the respective team. Next the CR is investigated and moved to state "Classified" pending approval by CCB or additional clarification is requested from a domain expert. If the CR is approved, it is updated to the state "To be Integrated" where required changes are planned for implementation and integration into a future FW build. Once a successful FW build with the requested implantation is available for testing, the state of the CR is updated to "Integrated". The CR is then redirected to the QA team for verification. The verifier then executes the CR workflow to confirm the defect is fixed and the linked test result passes. If verification is successful, the CR is set to state "Verified" and then archived by the tool else it is concluded that integrated changes fail, and approval is required to for further changes. If CR is not approved by CCB it is moved to "Terminated" state with a reasoning for the termination decision.

The roles responsible in the defect process are defined in the table below:

Responsible Role	Description
Submitter	QA engineer reporting observed defects during testing. it could also be a developer or other members of the product team
Investigator	Examination and classification of submitted CR based on severity and impact on release timeline
Domain Expert	Developer / Engineering expert with specialized focus area (e.g. FW, ASIC, Wireless)
Implementer / Integrator	Developer / Development team implementing the required changes and integrating it into a testable build
Verifier	QA engineer / team responsible for verifying the implemented changes into a testable build. Can not be the same person as the Implementer
Decider / CCB (Change Control Board)	Project Manager or Product Owner or Committee deciding whether changes should be implemented or the CR should be terminated based on performed investigations. Can not be the same person as the Submitter

The severity matrix for classifying defects is defined in the table below. The severity of a found defect is considered along with its likelihood to better assess the risk and impact on release timeline

Severity	Description
Safety Critical	High risk defect not handled safely causing harm to the user (e.g. FW fails to detect over-temperature or goes into undefined state where the thermal control of the system is silently lost)
High	Major functional defect causing the system to stop working or getting damaged (e.g. Failure to hold target temperature due to control loop divergence. It detects this and commands system shutdown)
Medium	Inconsistent behavior, performance issues and edge cases (e.g. high acoustic noise from the fan, fan speed marginally deviate from expected speed in the spec based on PWM signal, failure to hold temperature at peak load conditions)
Low	Minor issues with no impact on core system functionality (e.g. firmware does not correctly output log messages over UART)

The CR template contains the following fields which are filled and updated during its lifecycle:

- Title
- Reproduction steps
- Observed and expected behavior / workarounds if applicable
- HW revision
- FW build / version
- Investigation remarks / Investigator
- Implementation remarks / Implementer / Implementation pull request
- Verification remarks / Verifier
- Linked test case / test report logs and CI/CD run if applicable
- Severity and impact analysis results if applicable

4 Hardware In the Loop Test Setup

The proposed hardware in the loop test setup is depicted in figure (3) below. The Raspberry Pi is the test agent executing the automated python test cases. It communicates with the FW on the device under test via UART interface. The Raspberry Pi I2C interface could optionally be used to inject temperature data into FW. The Analog discovery connected to the Raspberry Pi via USB is used to capture the Tachometer PWM signal as well as commanded PWM output signal to the fan (Scope channels 1, 2). It is also used to supply power to the system during the test (V+).

By including the Analog Discovery, the test setup can explicitly measure the tachometer and output PWM signals and then compare them to the FW readings received over UART.

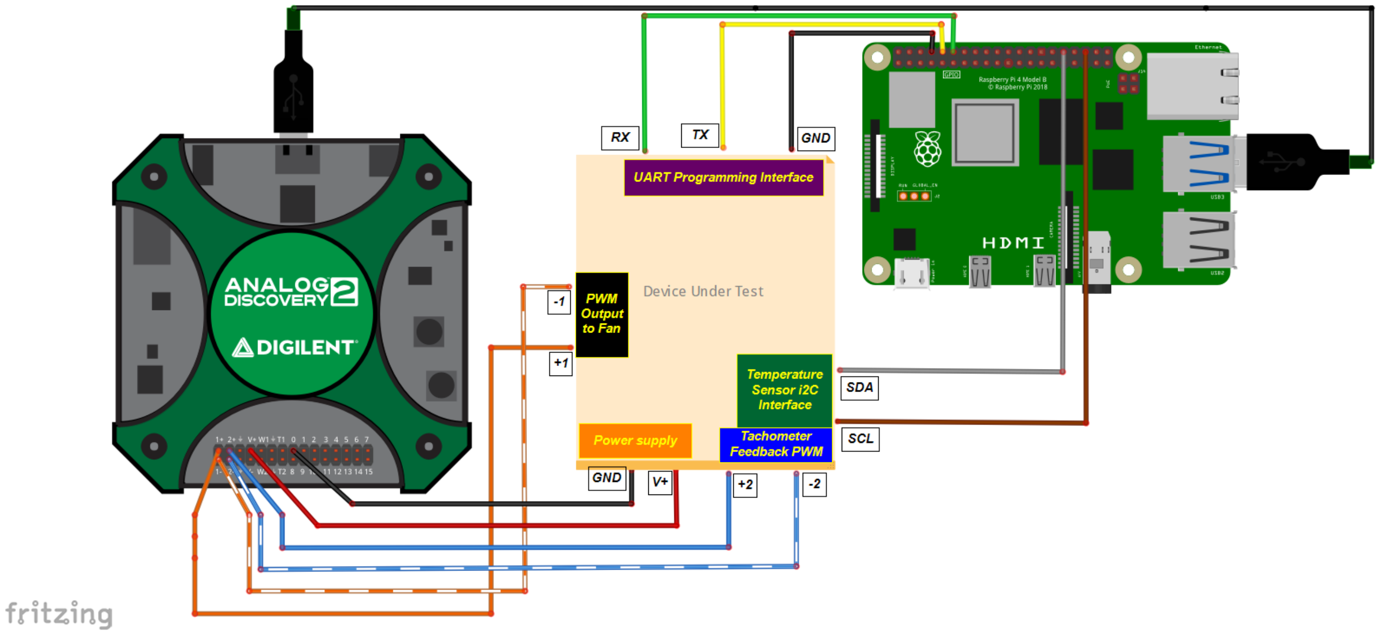

Figure 3: Test Setup Diagram and wiring to the Thermal control subsystem (hardware under test)

5 Risks and Mitigations

For a late-stage quality strategy, some of the expected project-related risks and mitigations are listed below:

Risk	Mitigation
Hardware workarounds and changes	Identify with the product team most representative hardware variants to test and execute the testing in regression on each variant to verify changes effectively
Late FW builds	Identify with the product team the possibility of intermediate test builds (e.g. release candidates) for bug fixes and changes where urgent testing feedback is required and execute regression test suite on official releases.
Unclear requirements and acceptance criteria	Early review of subsystem requirements can mitigate this. Product developers / architects are asked for clarification on missing input for the tests and include that input in revised version(s) of the requirements
FW configuration and hardware calibration not available	Request and agree on a certain FW configuration with the product team (e.g. target temperature, tolerances) that is as close to the end-user configuration as much as possible and ensure hardware is calibrated for production
Unstable test setup and test automation	Share the test setup documentation with the product team for review. Dedicate a small bring-up phase with the hardware and firmware developers to ensure required use cases are fulfilled and verified workarounds / fall back test methods exists for unstable and unsupported use cases)

6 Assumptions and Open Questions

- FW implements a programming interface over UART for communication with the device under test during hardware in the loop tests. The programming interface supports:
    - FW upload to target device (e.g. automated upload of new FW builds)
    - Setting control parameters (e.g. target temperature, temperature read interval, trim and calibration values for the PWM fan, control algorithm settings)
    - Getting status parameters from the FW (e.g. measured temperature, commanded PWM duty cycle, measured RPM speed from tachometer)
    - Test commands to inject faults (e.g. non-rotating fan, invalid temperature readings or I2C errors, drifting measured RPM speed)
- The tolerances used in the test scripts (e.g. temperature convergence, measured speed tracking) confirm with the datasheet of hardware components and subsystem requirements
- The fail-safe handling of major and safety critical defects (e.g. command 100% PWM duty cycle when temperature sensors fail, send shutdown request to thermal control dependent subsystems if the tachometer signal sense a non-rotating fan and /or over temperature is detected)
- The test setup and test framework are documented and validated
- FW developers execute a short smoke test suite prior to merging code changes (e.g. small CI/CD quality gate)
- The automated hardware-in-the loop tests are conducted in a room temperature environment (e.g. 25 degrees Celsius) and do not account for extreme environmental temperatures. It is assumed that reliability testing is carried out with the hardware under test being located inside a thermal chamber. The environment temperature will have impact on the performance of the subsystem components (e.g. temperature sensor ADC accuracy, MCU and PWM fan own temperature limits)